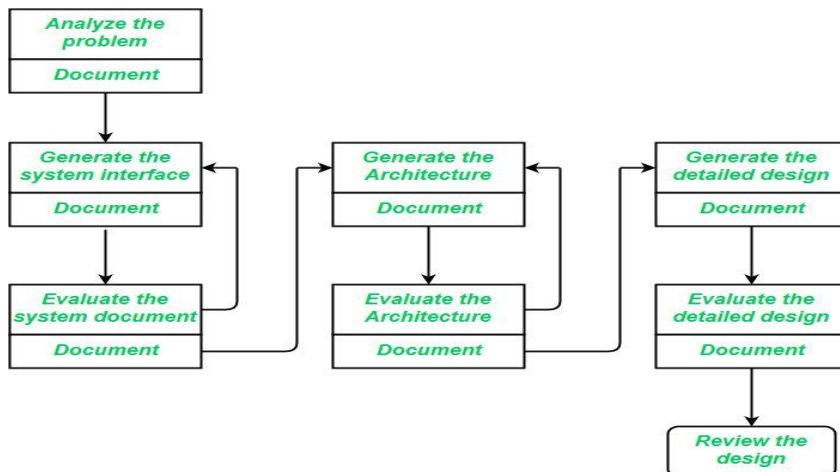


Unit- 3 Architectural Design

Design model:

- o The design phase of software development, transforming the customer requirements as described in the SRS documents into design forms.
- o Designing a model is an important phase and is a multi-process that represents the data structure, program structure, interface characteristic and procedural details.



Architectural Design.

- o Architectural design in software engineering is the process of creating the high and organization of a software system.
- o The architecture is the structure of program modules where they interact with each other in a specialized way. Or
- o It is also called as a blueprint for a system or product.

Structural Properties:

It involves :

1. Software Architecture: Overall design and structure.
2. System Components: Modules, subsystems, and interfaces.
3. Data Flow: Information exchange between components.
4. Deployment: Hardware and software environment.
5. Scalability: System's ability to handle increased load.

Architectural Views

It is generally used to represent entire architecture that is useful and meaningful to one or more stakeholders in system. It helps stakeholders understand the system's structure, behaviour, and interactions

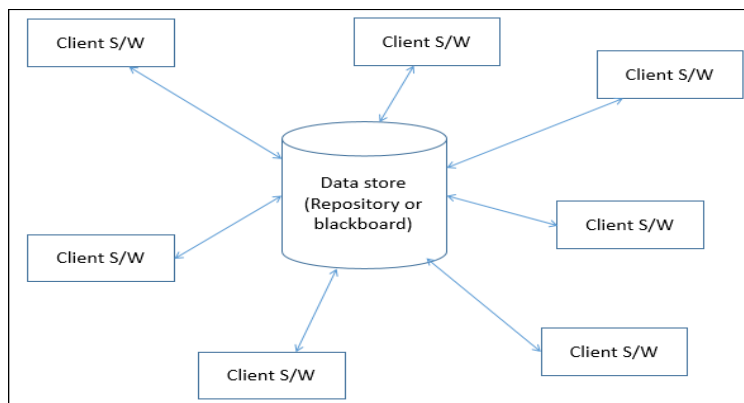
1. **Use case View:** Users view to handling product as per requirement. or Functional requirements Use case diagram is used.
2. **Design View:** Organize design information, significant features, entities and attributes.
3. **Process View:** Describe communication, behavior of the design.
4. **Implementation View:** It address source code integrators and developers of project.
5. **Deployment View:** It describes & explains environment within system runner & executed or Shows hardware and software deployment.

Tools and Techniques:

1. UML (Unified Modelling Language)
2. Graphical Representation (GR)

Architectural styles:

1] Data centered architectures:



➤ A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.

➤ The figure illustrates a typical data centered style. The client software access a central repository.

Variation of this approach are used to transform the repository into a blackboard when data related to client or data of interest for the client change the notifications to client software.

➤ This data-centered architecture will promote integrability. This means that the existing components can be changed and new client components can be added to the architecture without the permission or concern of other clients.

➤ Data can be passed among clients using blackboard mechanism.

Advantage of Data centered architecture:

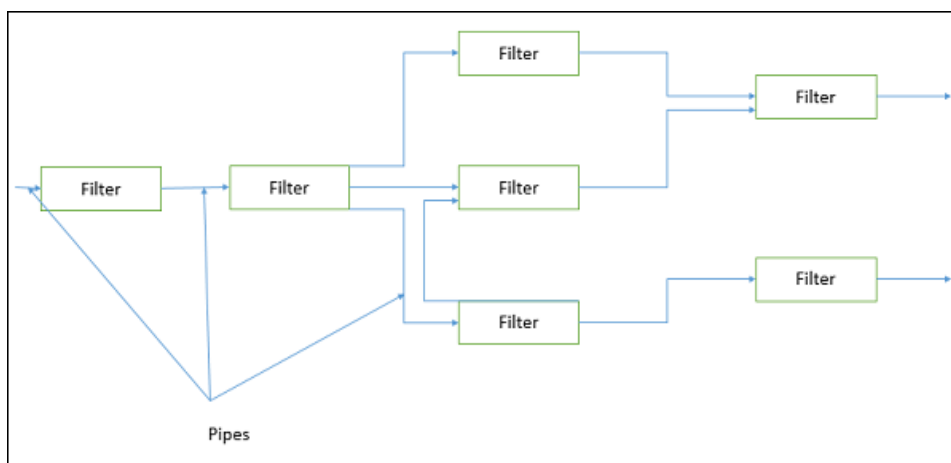
➤ Repository of data is independent of clients

➤ Client work independent of each other

➤ It may be simple to add additional clients.

➤ Modification can be very easy.

2] Data flow architectures:



➤ This kind of architecture is used when input data is transformed into output data through a series of computational manipulative components.

➤ The figure represents pipe-and-filter architecture since it uses both pipe and filter and it has a set of components called filters connected by lines.

➤ Pipes are used to transmitting data from one component to the next. ➤ Each filter will work independently and is designed to take data input of a certain form and produces data output to the next filter of a specified form. The filters don't require any knowledge of the working of neighboring filters.

➤ If the data flow degenerates into a single line of transforms, then it is termed as batch sequential. This structure accepts the batch of data and then applies a series of sequential components to transform it.

Advantages of Data Flow architecture

- It encourages upkeep, repurposing, and modification.
- With this design, concurrent execution is supported. The disadvantage of Data Flow architecture
- It frequently degenerates to batch sequential .
- Data flow architecture does not allow applications that require greater user engagement.
- It is not easy to coordinate two different but related streams.

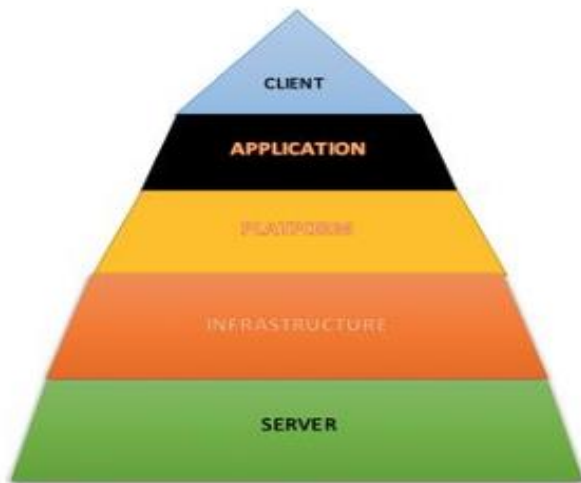
3]Object Oriented architecture:

The components of a system encapsulate data and the operations that must be applied to manipulate the data. The coordination and communication between the components are established via the message passing .

Characteristics of Object Oriented architecture

- Object protect the system's integrity.
- An object is unaware of the depiction of other items. Advantage of Object Oriented architecture
- It enables the designer to separate a challenge into a collection of autonomous objects.
- Other objects are aware of the implementation details of the object, allowing changes to be made without having an impact on other objects.

4] Layered architecture:



- A number of different layers are defined with each layer performing a well defined set of operations. Each layer will do some operations that becomes closer to machine instruction set progressively.
- At the outer layer, components will receive the user interface operations and at the inner layers, components will perform the operating system interfacing (communication and coordination with OS)
- Intermediate layers to utility services and application software functions.
- One common example of this architectural style is OSI-ISO (Open Systems Interconnection-International Organisation for Standardisation) communication system.

Architectural Patterns:

o The pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern

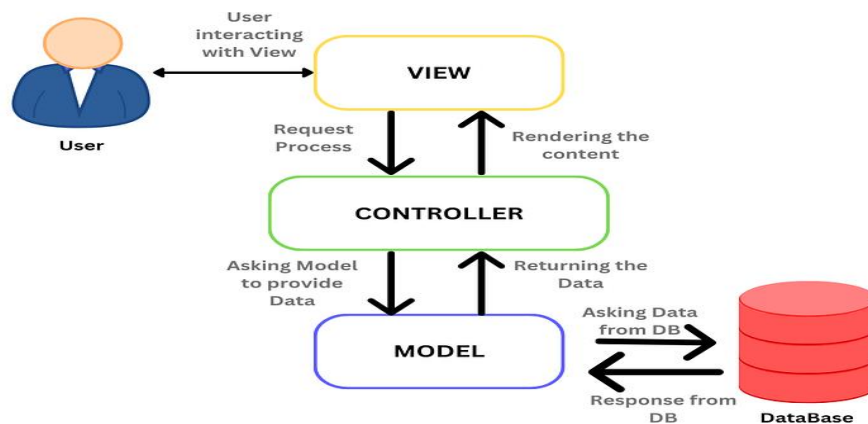
o Patterns are sequences that repeat based on rule.

Common architectural patterns are:

1. MVC (Model-View- Controller)
2. Layered,
3. Repository,
4. Client Server
5. Pipe and Filter System Modeling

1.MVC (Model-View-Controller):

- The Model View Controller (MVC) design pattern specifies that an application consists of a data model, presentation information, and control information. The pattern requires that each of these be separated into different Components.



The MVC pattern architecture consists of three layers:

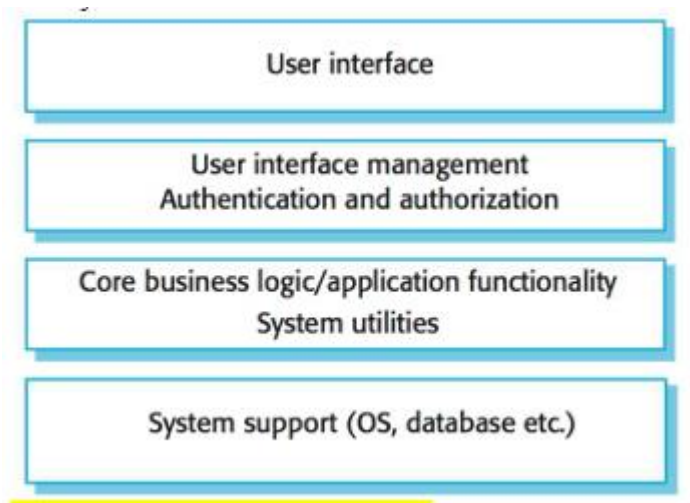
1. **Model:** It represents the business layer of application. It is an object to carry the data that can also contain the logic to update controller if data is changed.
2. **View:** It represents the presentation layer of application. It is used to visualize the data that the model contains.
3. **Controller:** It works on both the model and view. It is used to manage the flow of application, i.e. data flow in the model object and to update the view whenever data is changed.

Advantages of MVC Architecture

- o MVC has the feature of scalability that in turn helps the growth of application.
- o The components are easy to maintain because there is less dependency.
- o A model can be reused by multiple views that provide reusability of code.
- o The developers can work with the three layers (Model, View, and Controller) simultaneously.
- o Using MVC, the application becomes more understandable.
- o Using MVC, each layer is maintained separately therefore we do not require dealing with massive code.

2.Layered Architecture.

- Used to model the interfacing of sub-system.
- Organizes the system into a set of layers, each of which provides a set of services.
- When a layer interface changes, only the adjacent layer is affected.



Benefits of Layered Technology

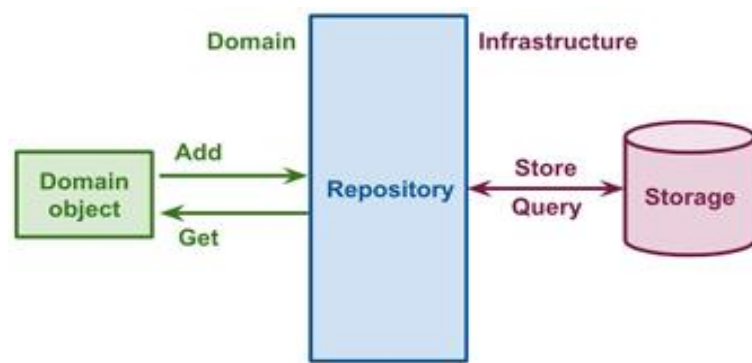
1. Modularity and Separation of Concerns
2. Scalability
3. Reusability
4. Interoperability
5. Testability
6. Maintainability

Implementing Layered Technology in Software Development

1. Choosing the Right Layers
2. Communication between Layers
3. Managing Dependencies
4. Handling Exceptions and Errors
5. Security Considerations
6. Scaling Layers
7. Monitoring and Debugging

3.Repository architecture.

- Repository architecture is a collection of independent components which operate on central data structure.
- In Repository Architecture Style, the data store is passive and the clients. (software components or agents) of the data store are active, which control the logic flow.



Advantages:

- Provides data integrity, backup and restore features.
- Provides scalability and reusability of agents as they do not have direct communication with each other.
- Reduces overhead of transient data between software components.

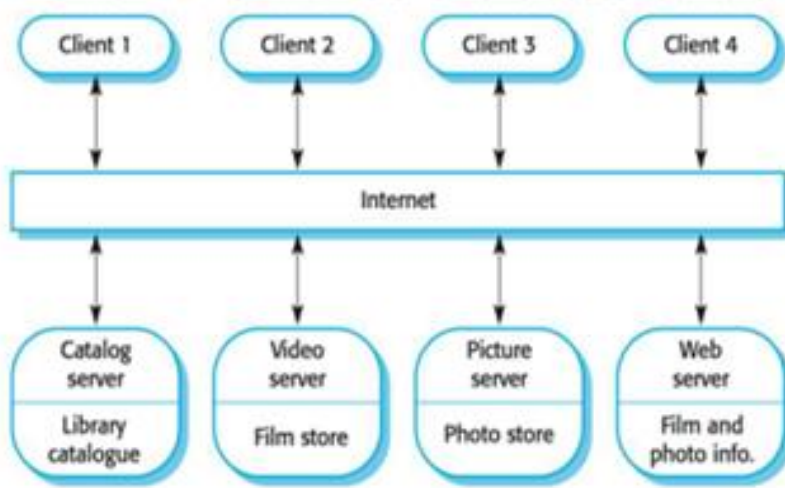
Disadvantages:

- o It is more vulnerable to failure and data replication or duplication is possible.
- o High dependency between data structure of data store and its agents.
- o Changes in data structure highly affect the clients.
- o Evolution of data is difficult and expensive.
- o Cost of moving data on network for distributed data.

4. Client-server architecture.

- o Distributed system model which shows how data and processing is distributed across a range of components, but can also be implemented on a single computer.
- o Set of clients which call on these services.
- o Network which allows clients to access servers.

Usage: Online applications such as email, document sharing and banking.



Key Components of Client Server Architecture:

client architecture in system design involves several key components that work together to ensure efficient communication, resource management, and Client-server architecture in system design involves several key components that work together to ensure efficient communication, resource management, and service delivery.

Here are the main components:

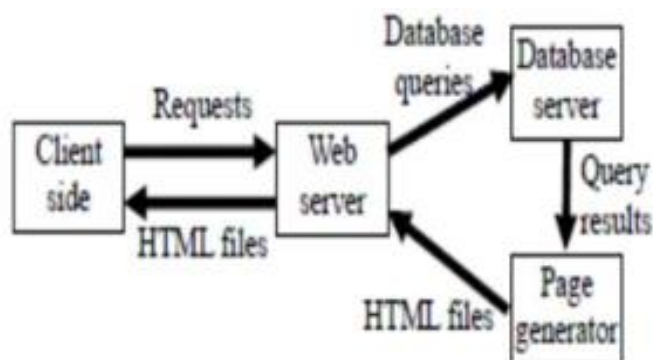
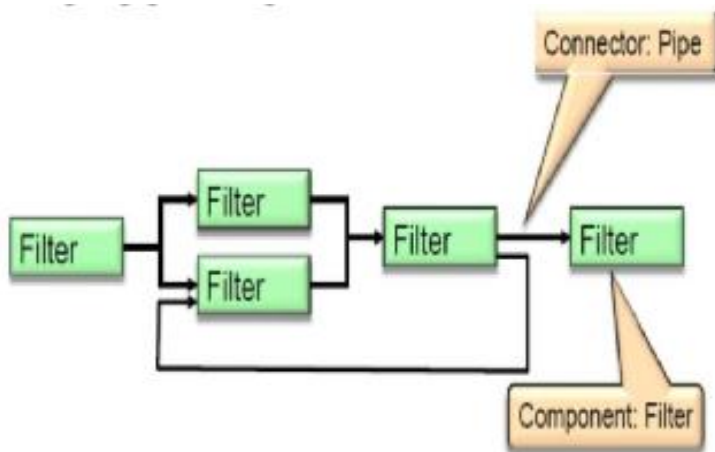
1. Client: The client is a device or application that requests services or resources from the server.
2. Server: The server is a powerful machine or application that provides services or resources to clients.
3. Network: The network The communication between clients and servers.
4. Protocols: Protocols are standardized rules that govern data transmission and communication between clients and servers.
5. Middleware: Middleware is software that acts as an intermediary between clients and servers.
6. Database: A database is a structured collection of data stored on the server.
7. User Interface (UI): The UI is the part of the client application that interacts with the user

5. Pipe and Filter System Modeling.

o This architecture is used when input data to be transformed into output data through a series of computational manipulative components.

o Pipe: is a connector which passes the data directionally from one filter to the next.

o Filter: is a component reads the data from its input pipes and performs its function.



Benefits of Pipe and Filter Architecture

The Pipe and Filter architecture offers numerous benefits that make it an attractive choice for designing complex systems.

Here are some of the key benefits:

1. Enhanced Data Processing:
2. Ease of Understanding:
3. Isolation of Faults:
4. Improved Testing:
5. Resource Optimization:

System modelling:

System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers. Different models present the system from different perspectives

- External perspective showing the system's context or environment;
- Behavioural perspective showing the behaviour of the system;
- Structural perspective showing the system or data architecture.

Interaction models:

- Modeling user interaction is important as it helps to identify user requirements.
- Modeling system-to-system interaction highlights the communication problems that may arise.
- Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.
- Use case diagrams and sequence diagrams may be used for interaction modelling.
- They are used to visualize the flow of interactions within a system, showing how different parts of the system communicate and collaborate to achieve a specific goal.

1. Use case diagrams.

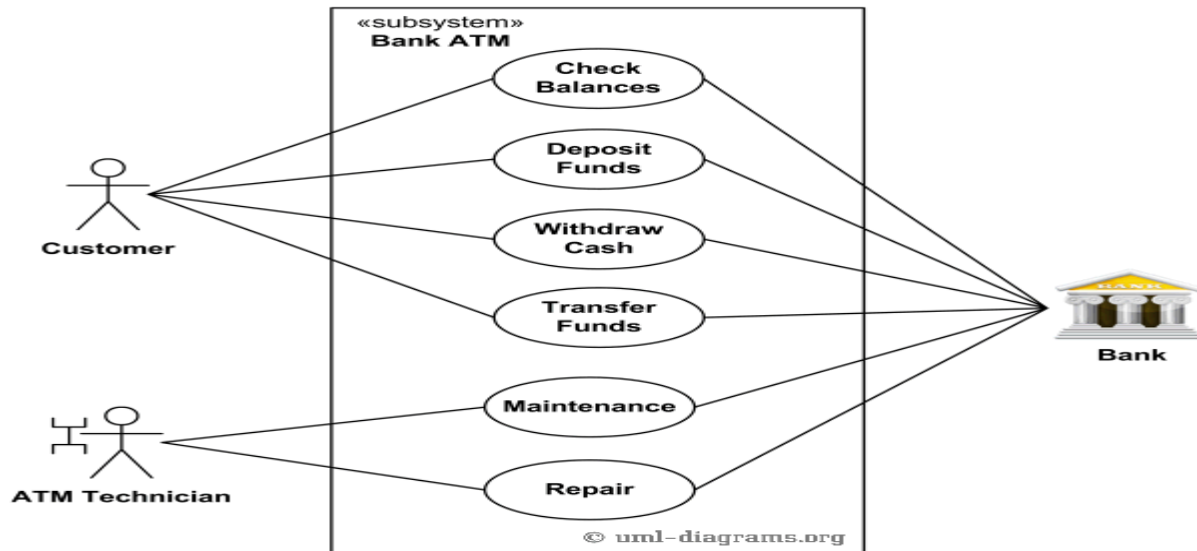
- o The purpose of a use case diagram in UML is to demonstrate the different ways that a user might interact with a system.
- o Create a professional diagram for nearly any use case using our UML diagram tool.
- o Each use case represents a discrete task that involves external interaction with a system.
- o Use cases were developed originally to support requirements elicitation and now incorporated into the UML.



BANK ATM Use Case Diagram

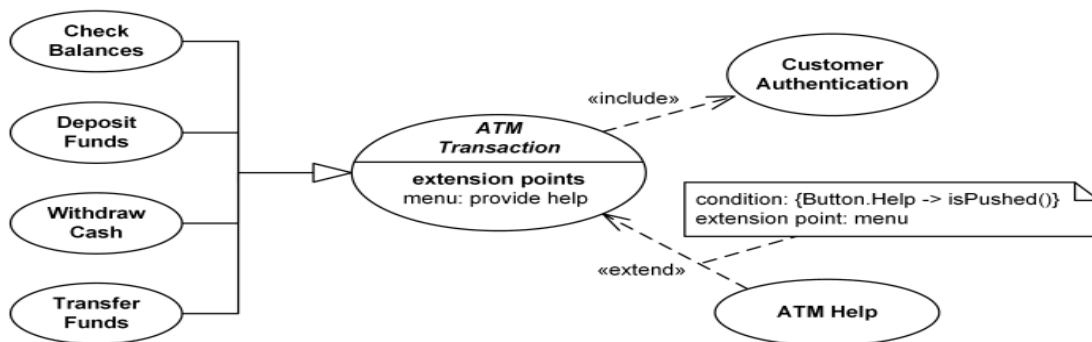
An automated teller machine (**ATM**) or the automatic banking machine (**ABM**) is a banking subsystem (**subject**) that provides bank customers with access to financial transactions in a public space without the need for a cashier, clerk, or bank teller.

Customer (**actor**) uses bank ATM to *Check Balances* of his/her bank accounts, *Deposit Funds*, *Withdraw Cash* and/or *Transfer Funds* (**use cases**). ATM Technician provides *Maintenance* and *Repairs*. All these use cases also involve *Bank* actor whether it is related to customer transactions or to the ATM servicing.



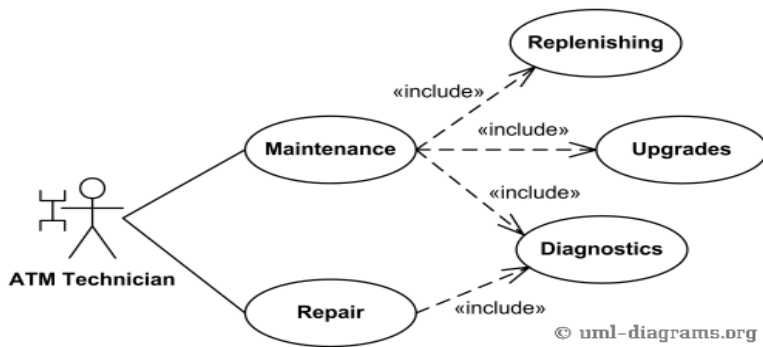
An example of use case diagram for Bank ATM subsystem - top level use cases

On most bank ATMs, the customer is authenticated by inserting a plastic ATM card and entering a personal identification number (PIN). *Customer Authentication* use case is required for every ATM transaction so we show it as **include** relationship. Including this use case as well as transaction **generalizations** make the *ATM Transaction* an **abstract use case**.



Bank ATM Transactions and Customer Authentication Use Cases Example.

Customer may need some help from the ATM. *ATM Transaction* use case is **extended** via **extension point** called *menu* by the *ATM Help* use case whenever *ATM Transaction* is at the location specified by the *menu* and the bank customer requests help, e.g. by selecting Help menu item.



Bank ATM Maintenance, Repair, Diagnostics Use Cases Example.

ATM Technician maintains or repairs Bank ATM. *Maintenance* use case includes *Replenishing* ATM with cash, ink or printer paper, *Upgrades* of hardware, firmware or software, and remote or on-site *Diagnostics*. *Diagnostics* is also **included** in (shared with) *Repair* use case.

Use cases: Horizontally shaped ovals that represent the different uses that a user might have.

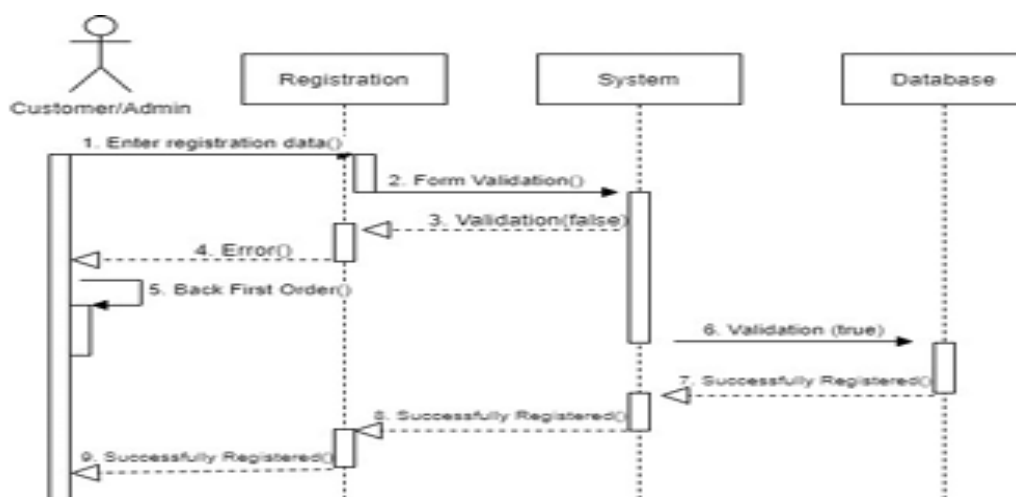
Actors: Stick figures that represent the people actually employing the use cases.

Associations: A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

2. Sequence diagrams.

- The sequence diagram represents the flow of messages in the system.
- The communication between any two lifelines as a time-ordered sequence of events.
- The lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page.
- Sequence diagrams are part of the UML and are used to model the interactions between the actors and the objects within a system.
- A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.

Sequence diagrams of registration page



1. Enter the registration data.
2. Form validation.
3. Form validation is false.
4. Display the Error on user interface.
5. Back to the first order.
6. Validation is true.
7. Successfully registered sent to the system.
8. Successfully registered sent to the registration form.
9. Successfully registered finally shown to the user.

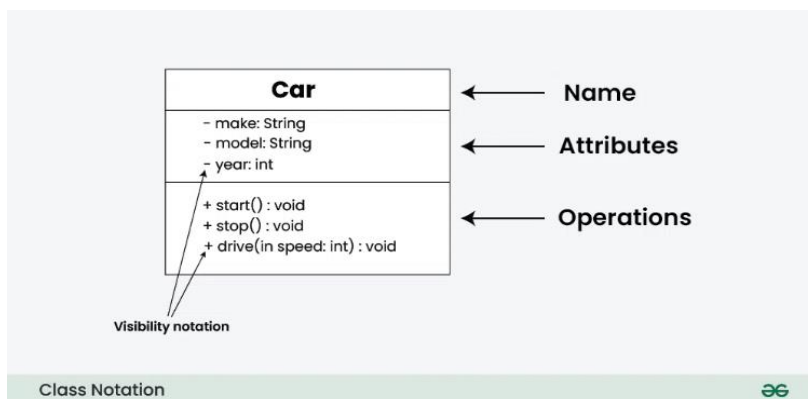
Structural Modeling: Class diagrams:

Structural models:

- Structural models of software display the organization of a system in terms of the components that make up that system and their relationships.
- Structural models may be static models, which show the structure of the system design, or dynamic models, which show the organization of the system when it is executing. You create structural models of a system when you are discussing and designing the system architecture

Class diagrams:

- Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.
- An object class can be thought of as a general definition of one kind of system object.
- An association is a link between classes that indicates that there is some relationship between these classes.



1. Class Name:

- The name of the class is typically written in the top compartment of the class box and is centered and bold.

2. Attributes:

- Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

3. Methods:

- Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

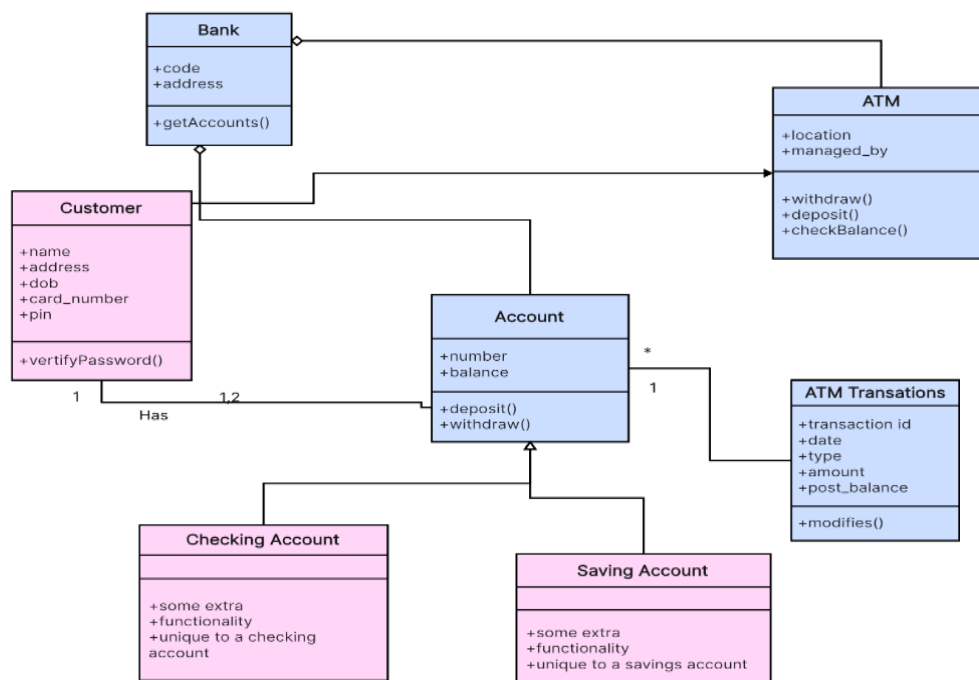
4. Visibility Notation:

- Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
 - + for public (visible to all classes)
 - for private (visible only within the class)
 - # for protected (visible to subclasses)
 - ~ for package or default visibility (visible to classes in the same package)

Class diagram for Banking System

- Customer:** This class could have attributes like customer_id, name, address, and contact_number. Methods could include create_account(), request_loan().
- Account:** Attributes could include account_number, customer_id, balance, account_type. It could have operations like debit(), credit(), check_balance().
- Loan:** The loan class could have loan_id, customer_id, amount, interest_rate as attributes, and approve_loan(), calculate_interest() as methods.
- Transaction:** Its attributes might be transaction_id, account_number, transaction_type, amount, date.

Banking System



Key points:

- A model is an abstract view of a system that ignores system details. Complementary system models can be developed to show the system's context, interactions, structure and behavior.
- Context models show how a system that is being modeled is positioned in an environment with other systems and processes.
- Use case diagrams and sequence diagrams are used to describe the interactions between users and systems in the system being designed. Use cases describe interactions between a system and external actors; sequence diagrams add more information to these by showing interactions between system objects.
- Structural models show the organization and architecture of a system. Class diagrams are used to define the static structure of classes in a system and their associations.

Relationships: A generalization is a relationship between a parent class (super class) and a child class (subclass). In this, the child class is inherited from the parent class. For example, The Current Account, Saving Account, and Credit Account are the generalized form of Bank Account.

Behavioral Modeling:

Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment.

You can think of these stimulus as being of two types: –

Data Some data arrives that has to be processed by the system.

Events Some event happens that triggers system processing. Events may have associated data, although this is not always the case

o It involves analyzing and representing the dynamic aspects of a system, such as...

1. User interactions
2. System responses
3. Data flow
4. Control flow
5. Concurrent processes

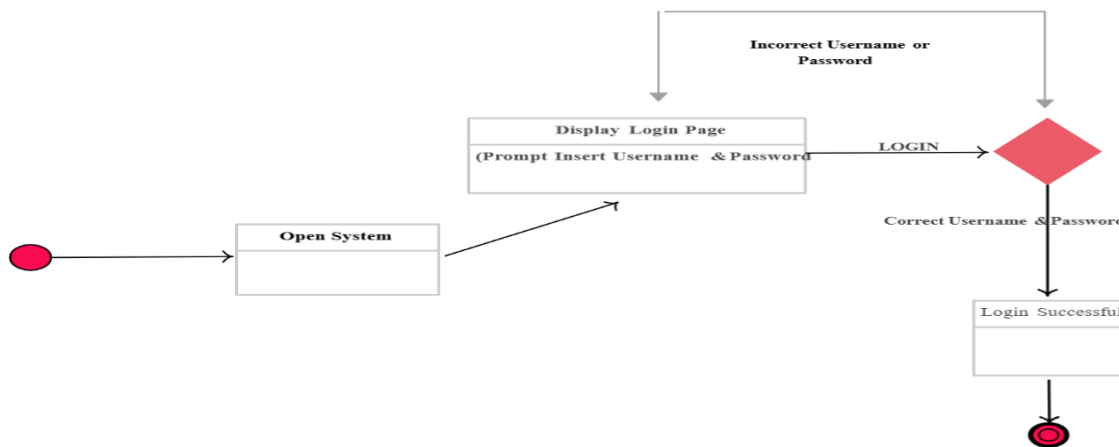
State diagrams:

o A state diagram is a behavioral modeling technique used to describe the dynamic behavior of a system, component, or object.

o It represents the various states an entity can be in and the transitions between those states.

- These model the behaviour of the system in response to external and internal events.
- They show the system's responses to stimuli so are often used for modelling real-time systems.

- State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another.
 - Statecharts are an integral part of the UML and are used to represent state machine models.
- o State diagram of login page



Components:

1. States (nodes): Represent specific situations or conditions.
2. Transitions (arrows): Describe changes between states.
3. Events (labels): Trigger transitions.
4. Guards (conditions): Control transition execution.
5. Actions (activities): Perform tasks during transitions.

Functional Modeling: Data flow diagrams.

o Functional modeling is a software engineering technique used to represent the functional requirements of a system, focusing on the inputs, processing, outputs, and interactions between components.

Data Flow Diagram (DFD): in Software Engineering a Data Flow Diagram (DFD) is a graphical representation of the flow of data through a system, highlighting inputs, processing, outputs, and storage.

- It shows how data enters and leaves the system, what changes the information, and where data is stored.

DFD Components:

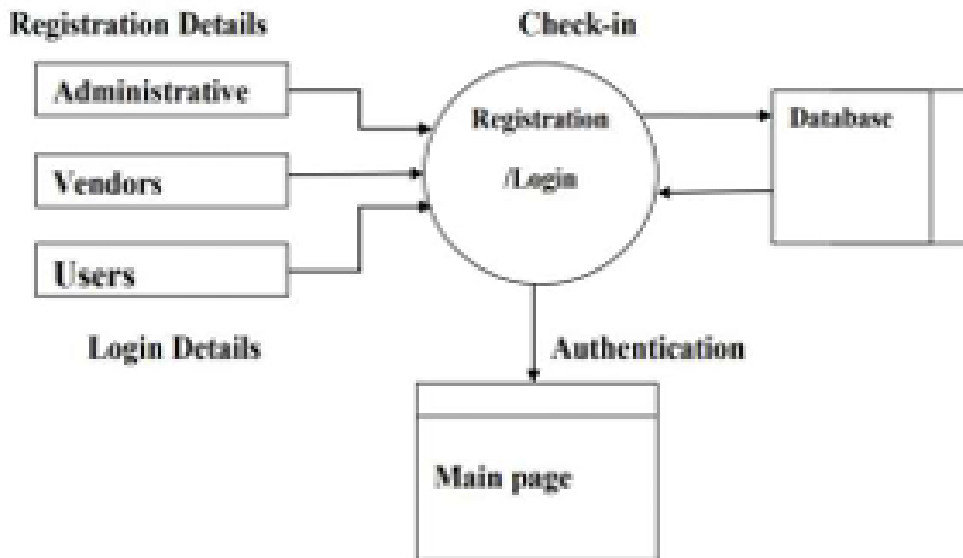
1. Processes (Bubbles): Represent transformations or operations.
2. Data Flows (Arrows): Show data movement between processes, data stores, or external entities.
3. Data Stores (Open-ended Rectangles): Represent data repositories.
4. External Entities (Rectangles): Represent sources or destinations of data.
5. Context (System Boundary): Defines the system's scope.

DFD Levels:

1. Context Diagram (Level 0): High-level overview of the system.

o A context diagram is a top level (also known as "Level 0") data flow diagram.

o It only contains one process node ("Process 0") that generalizes the function of the entire system in relationship to external entities.



2. Logical Data Flow Diagram (Level 1): Detailed representation of data flow.

o The Level 1 DFD shows how the system is divided into sub-systems (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole.



3. **Physical Data Flow Diagram (Level 2): Represents actual implementation.**

- o A DFD that represents a decomposed level 1 DFD is called a level 2 DFD. There can be a level 2 DFD for each process that appears in the level 1 DFD.

